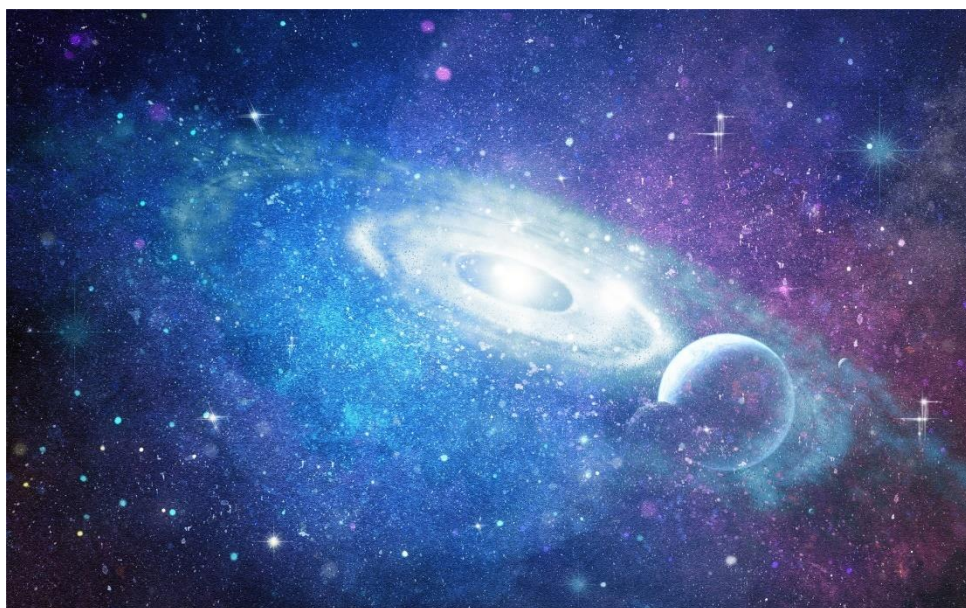


算法设计与分析实验教程 (C++版)



张德福、曾华琳、李胜睿、卢杨、郝宗寅等编著

《算法设计与分析》教研组

厦门大学信息学院计算机系、厦门大学信息学院实验教学中心

目录

目录	I
第1章 算法设计与分析实验A	1
1.1 装配线调度问题	1
1.2 矩阵链乘法问题	3
1.3 最长公共子序列	4
1.4 01背包问题	6
1.5 最短Hamilton路径	7
1.6 最小编辑代价	9
第2章 算法设计与分析实验B	11
2.1 多多合并果果	11
2.2 小H的扑克游戏	12
2.3 看直播	14
2.4 多多建设农场	15
2.5 多多维持治安	17
2.6 小H的选课计划	19
第3章 算法设计与分析实验C	23
3.1 小H保护小草	23
3.2 小H的导师选择	25
3.3 少少种谷物	28
3.4 多多的棋盘游戏	32
3.5 整数划分	35
第4章 算法设计与分析实验D	37
4.1 最短编辑距离	37
4.2 小H的塔防游戏	38
4.3 小H的塔防游戏2	40
4.4 小H的超大背包	42
4.5 小H的旅行计划	43
4.6 小H的旅行计划2	45
后记	48

第1章 算法设计与分析实验A

1.1 装配线调度问题

描述

有两条装配线，每一条装配线上有 n 个装配点，将装配线 i 上的第 j 个装配点记为 $S_i[j]$ ，设在装配点 $S_i[j]$ 的装配时间为 $a_i[j]$ 。假设要装配一辆汽车，将汽车底盘从进厂点送入装配线 i ($i = 1$ 或 2)，需要时间 e_i 。在装配点 $S_i[j]$ 装配后，如果汽车传送到同一条装配线的装配点 $S_i[j+1]$ 进行装配，则传送不需要时间。如果汽车从装配点 $S_i[j]$ 传送到另一条装配线进行装配，则需要传送时间 $t_i[j]$ 。汽车在装配点 $S_i[n]$ 装配后，将汽车成品从装配线上取下来，需要花费时间 x_i 。装配线调度问题是如何确定每一个装配点的装配需要在哪条线上进行，使得当汽车成品出来时，花费的总时间最少。值得注意的是两条装配线的第 j 个装配点都装配同样的汽车部件，只是装配效率不一样。

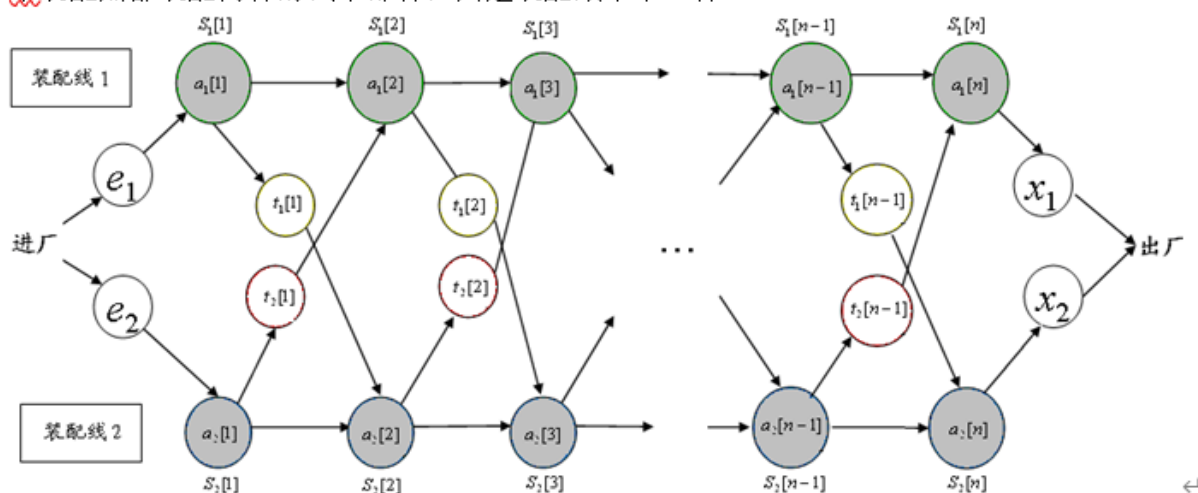


图6.2

输入

输入共 7 行

第一行一个数字，表示节点数 $n[2, 10000]$

第二行 n 个数字依次表示第一条流水线处理时间 $[1, 100]$

第三行 n 个数字依次表示第二条流水线处理时间 $[1, 100]$

第四行n-1个数字依次表示第一条流水线转至第二条流水线消耗时间[1,100]

第五行n-1个数字依次表示第二条流水线转至第一条流水线消耗时间[1,100]

第六行2个数字,分别表示送入流水线1和流水线2的入厂时间[1,100]

第七行2个数字,分别表示离开流水线1和流水线2的出场时间[1,100]

输出

输出最短时间

输入样例 1

```
6
7 9 3 4 8 4
8 5 6 4 5 7
2 3 1 3 4
2 1 2 2 1
2 4
3 2
```

输出样例 1

```
38
```

提示

来源：张德富，《算法设计与分析》P68：6.2

算法思路

```
1. #include <algorithm>
2. #include <iostream>
3. using namespace std;
4. const int N = 1110, M = 3000;
5. int a[N], b[N], f[M][M];
6. int main() {
7.     int n;
8.     cin >> n;
9.     a[0] = 1;
10.    for (int j = 0; j < n; ++j) cin >> a[j] >> b[j];
11.    a[n] = b[n - 1];
12.    for (int r = 2; r <= n; ++r) {
13.        for (int i = 1; i <= n - r + 1; ++i) {
14.            int j = r + i - 1;
15.            f[i][j] = f[i][i] + f[i + 1][j] + a[i - 1] * a[i] * a[j];
16.            for (int k = i + 1; k < j; ++k)
17.                f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j] + a[i - 1] * a[k] * a[j]);
18.        }
```

```

19.  }
20.  cout << f[1][n] << endl;
21.  return 0;
22. }

```

1.2 矩阵链乘法问题

描述

给定一个有 n 个矩阵的矩阵链(Matrix Chain) $A_1, A_2, \dots, A_i, \dots, A_n$, 矩阵 A_i ($1 \leq i \leq n$)

的维数为 $p_{i-1} \times p_i$, 矩阵链乘法问题就是如何对矩阵乘积 $A_1 A_2 \cdots A_i \cdots A_n$ 加括号, 使得它们的标量乘法次数达到最少。矩阵乘积可以加括号, 指的是矩阵的乘积是可以完全括号化的, 即要么是单一的矩阵要么是两个完全括号化的矩阵相乘之后再加上括号而形成的。↵

输入

输入共 $n+1$ 行

第一行输入矩阵的总个数 $n[2, 1000]$

后 n 行分别输入矩阵的维数 $[1, 100]$

输出

最后一行输出少乘法次数

输入样例 1

```

6
30 35
35 15
15 5
5 10
10 20
20 25

```

输出样例 1

```

15125

```

提示

来源: 《算法设计与分析》 - 张德富

算法思路

```
1. #include <algorithm>
2. #include <iostream>
3. using namespace std;
4. const int N = 1110, M = 3000;
5. int a[N], b[N], f[M][M];
6. int main() {
7.     int n;
8.     cin >> n;
9.     a[0] = 1;
10.    for (int j = 0; j < n; ++j) cin >> a[j] >> b[j];
11.    a[n] = b[n - 1];
12.    for (int r = 2; r <= n; ++r) {
13.        for (int i = 1; i <= n - r + 1; ++i) {
14.            int j = r + i - 1;
15.            f[i][j] = f[i][i] + f[i + 1][j] + a[i - 1] * a[i] * a[j];
16.            for (int k = i + 1; k < j; ++k)
17.                f[i][j] = min(f[i][j], f[i][k] + f[k + 1][j] + a[i - 1] * a[k] * a[j]);
18.        }
19.    }
20.    cout << f[1][n] << endl;
21.    return 0;
22. }
```

1.3 最长公共子序列

描述

在生物工程应用中，我们经常要比较两个 DNA 序列的相似性，在软件工程领域也经常比较两个软件版本，以便确定版本的变化，所有这些相似性比较问题，都涉及最长公共子序列问题(Longest Common Subsequence, 简称 LCS)。为设计该问题的求解算法，我们先给出一些概念。^①

一个给定序列的子序列是在该序列中删去若干元素后得到的序列，形式的定义如下^②

给定一个序列 $X = \langle x_1, x_2, \dots, x_m \rangle$ ，另一个序列 $Z = \langle z_1, z_2, \dots, z_k \rangle$ 是 X 的子序列，

如果存在一个严格递增的 X 中元素下标的序列 $\langle i_1, i_2, \dots, i_k \rangle$ ，使得 $x_{i_j} = z_j (1 \leq j \leq k)$ 。^③

给定两个序列 X 和 Y ，当另一序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的公共子序列。最长公共子序列就是公共子序列中长度最长的子序列。^④

请给出代码求两字符串最长公共子序列。

输入

输入共三行

第一行是一个数 $n[2, 1000]$, 代表两个字符串的长度

接下来两行分别为两待求字符串['A', 'Z']

输出

输出最长公共子序列长度

输入样例 1

```
7
ABCBDAAB
BDCABAZ
```

输出样例 1

```
4
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int N;
5.
6. int f[1005][1005];
7. char u[1005], v[1005];
8.
9. int main() {
10.     scanf("%d%s%s", &N, u + 1, v + 1);
11.
12.     for (int i = 1; i <= N; i++)
13.         for (int j = 1; j <= N; j++)
14.             if (u[i] == v[j])
15.                 f[i][j] = f[i - 1][j - 1] + 1;
16.             else
17.                 f[i][j] = max(f[i - 1][j], f[i][j - 1]);
18.
19.     printf("%d\n", f[N][N]);
20.
21.     return 0;
22. }
```

1.4 01背包问题

描述

有 N 件物品和一个容量是 V 的背包。每件物品只能使用一次。

第 i 件物品的体积是 v_i ，价值是 w_i 。

求解将哪些物品装入背包，可使这些物品的总体积不超过背包容量，且总价值最大。

输出最大价值。

数据范围

$0 < N, V \leq 1000$

$0 < v_i, w_i \leq 100$

输入

第一行两个整数， N ， V ，用空格隔开，分别表示物品数量和背包容积。

接下来有 N 行，每行两个整数 v_i, w_i ，用空格隔开，分别表示第 i 件物品的体积和价值。

输出

输出一个整数，表示最大价值。

输入样例 1

```
4 5
1 2
2 4
3 4
4 5
```

输出样例 1

```
8
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int N, M;
5.
6. int f[1005];
7. int s[1010][2];
8.
9. int main() {
```

```

10. scanf("%d%d", &N, &M);
11. for (int i = 1; i <= N; i++) scanf("%d%d", &s[i][0], &s[i][1]);
12.
13. for (int k = 1; k <= N; k++)
14.     for (int v = M; v >= s[k][0]; v--)
15.         f[v] = max(f[v], f[v - s[k][0]] + s[k][1]);
16.
17. int ans = 0;
18. for (int v = M; v; v--) ans = max(ans, f[v]);
19.
20. printf("%d\n", ans);
21.
22. return 0;
23. }

```

1.5 最短Hamilton路径

描述

给定一张 n 个点的带权无向图，点从 $0 \sim n-1$ 标号，求起点 0 到终点 $n-1$ 的最短Hamilton路径。Hamilton路径的定义是从 0 到 $n-1$ 不重不漏地经过每个点恰好一次。

数据范围

$$1 \leq n \leq 20$$

$$0 \leq a[i, j] \leq 10^7$$

输入

第一行输入整数 n 。

接下来 n 行每行 n 个整数，其中第 i 行第 j 个整数表示点 i 到 j 的距离（记为 $a[i, j]$ ）。

对于任意的 x, y, z ，数据保证 $a[x, x]=0$ ， $a[x, y]=a[y, x]$ 并且 $a[x, y]+a[y, z] \geq a[x, z]$ 。

输出

输出一个整数，表示最短Hamilton路径的长度。

输入样例 1

```

5
0 2 4 5 1
2 0 6 5 3
4 6 0 8 3

```

```
5 5 8 0 5
1 3 3 5 0
```

输出样例 1

```
18
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3. template <typename AA>
4. inline void CMin(AA &u, AA v) {
5.     if (v < u) u = v;
6. }
7.
8. int N, M;
9.
10. int f[20][1 << 20], top, m[20][20];
11.
12. int main() {
13.     scanf("%d", &N);
14.     for (int a = 0; a < N; a++)
15.         for (int b = 0; b < N; b++) scanf("%d", &m[a][b]);
16.
17.     memset(f, 0x3f, sizeof(f));
18.     f[0][1] = 0;
19.     top = 1 << N;
20.
21.     for (int k = 3; k < top; k += 2)
22.         for (int p = 1; p < N; p++)
23.             if (k & (1 << p)) {
24.                 for (int i = 0; i < N; i++)
25.                     if ((i != p) && (k & (1 << i))) {
26.                         CMin(f[p][k], f[i][k ^ (1 << p)] + m[i][p]);
27.                         // printf("%d %d %d\n", p, k, f[p][k]);
28.                     }
29.             }
30.
31.     printf("%d\n", f[N - 1][top - 1]);
32.
33.     return 0;
34. }
```

1.6 最小编辑代价

描述

给定两个字符串`str1`和`str2`，再给定三个整数`c0`，`c1`和`c2`，分别代表插入、删除和替换一个字符的代价，请输出将`str1`编辑成`str2`的最小代价。

输入

第一行：字符串`str1`，长度`[1,1500]`

第二行：字符串`str2`

第三行：三个整数`c0`，`c1`，`c2`

输出

最小编辑代价

输入样例 1

```
abc
adc
5 3 2
```

输出样例 1

```
2
```

输入样例 2

```
abc
adc
5 3 100
```

输出样例 2

```
8
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3. template <typename AA>
4. inline void CMin(AA &u, AA v) {
5.     if (v < u) u = v;
6. }
7.
8. int N, M;
```

```

9.
10. int f[1505][1505];
11. char u[1505], v[1505]; // u to v
12. int i, e, r;
13.
14. int main() {
15.     cin >> u + 1 >> v + 1 >> i >> e >> r;
16.     int lu = strlen(u + 1), lv = strlen(v + 1);
17.
18.     memset(f, 0x3f, sizeof(f));
19.     f[0][0] = 0;
20.     for (int a = 1; a <= lu; a++) f[a][0] = e * a;
21.     for (int b = 1; b <= lv; b++) f[0][b] = i * b;
22.
23.     for (int a = 1; a <= lu; a++)
24.         for (int b = 1; b <= lv; b++) {
25.             if (u[a] == v[b])
26.                 CMin(f[a][b], f[a - 1][b - 1]);
27.             else
28.                 CMin(f[a][b], f[a - 1][b - 1] + r);
29.             CMin(f[a][b], f[a][b - 1] + i);
30.             CMin(f[a][b], f[a - 1][b] + e);
31.         }
32.     cout << f[lu][lv] << endl;
33.
34.     return 0;
35. }

```

第2章 算法设计与分析实验B

2.1 多多合并果果

描述

在一个果园里，多多已经把所有的果子打了下来，而且按果子的不同种类分成了不同的堆。多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子的重量之和。可以看出，所有的果子经过 $n-1$ 次合并之后，就只剩下一堆了。多多在合并果子时总共消耗的体力等于每次合并所消耗体力之和。

因为还要花大力气把这些果子搬回家，所以多多在合并果子时要尽可能地节省体力。假定每个果子重量都为1，并且已知果子的种类数和每种果子的数目，你的任务是设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

例如有3种果子，数目依次为1，2，9。可以先将1、2堆合并，新堆数目为3，耗费体力为3。接着，将新堆与原先的第三堆合并，又得到新的堆，数目为12，耗费体力为12。所以多多总共耗费体力 $=3+12=15$ 。可以证明15为最小的体力耗费值。

输入

第一行是一个整数 n ($1 \leq n \leq 10000$)，表示果子的种类数。

第二行包含 n 个整数，用空格分隔，第 i 个 a_i ($1 \leq a_i \leq 20000$) 是第 i 个果子的数目。

输出

输出包括一行，这一行只包含一个整数，也就是最小的体力耗费值。输入数据保证这个值小于 2^{31} 。

输入样例 1

```
3
1 2 9
```

输出样例 1

```
15
```

算法参考

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. priority_queue<int, vector<int>, greater<int> > q;
5. int N;
6.
```

```

7. int main() {
8.     scanf("%d", &N);
9.     for (int a = 1; a <= N; a++) {
10.        int t;
11.        scanf("%d", &t);
12.        q.push(t);
13.    }
14.    int ans = 0;
15.    while (q.size() > 1) {
16.        int l, r;
17.        l = q.top();
18.        q.pop();
19.        r = q.top();
20.        q.pop();
21.        ans += l + r;
22.        q.push(l + r);
23.    }
24.    printf("%d\n", ans);
25.
26.    return 0;
27. }

```

2.2 小H的扑克游戏

描述

小H和小Z玩扑克游戏。两人从牌堆中各抽取 n 张牌，然后每次从手牌中选取一张牌比较大小：

1. 当小H的牌大于小Z的牌时，小H赢得10枚代币。
2. 当小H的牌小于小Z的牌时，小H输掉10枚代币。
3. 若两人的牌相同，则不输不赢。

牌堆中牌的大小顺序为：大王>小王>A>K>Q>J>10>9>.....>3>2。且牌堆中每种牌的数量无限。

现小H使用了一种高科技眼镜，能够知道小Z抽到的每一张牌的大小以及小Z每次出牌的大小。

问：小H采取怎样的策略，能够赢得最多的代币？

输入

输入数据包含多个测试实例：

每个测试实例的第一行有一个整数 n ，表示从牌堆中抽牌的数量 n 。（ $1 \leq n \leq 1000$ ）

第二行包括 n 个整数，表示小H抽到的每张牌的大小。（11代表J，12代表Q，13代表K，14代表A，15代表小王，16代表大王）

第三行包括 n 个整数，表示小Z抽到的每张牌的大小。

输出

对于每个测试实例，输出小_H最多能赢的代币数。每个测试实例的输出占一行。

输入样例 1

```
4
9 6 4 12
13 10 8 8
2
16 13
2 5
2
3 5
14 11
```

输出样例 1

```
0
20
-20
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int N;
5. int s[1010], t[1010];
6.
7. int main() {
8.
9.     while (cin >> N) {
10.         int ans = 0;
11.         for (int a = 1; a <= N; a++) cin >> s[a];
12.         for (int a = 1; a <= N; a++) cin >> t[a];
13.         sort(s + 1, s + 1 + N);
14.         sort(t + 1, t + 1 + N);
15.
16.         int sh = 1, th = 1, st = N, tt = N;
17.         for (int ttt = 1; ttt <= N; ttt++) {
18.             if (s[sh] > t[th]) {
19.                 ans += 10;
20.                 sh++, th++;
21.             } else if (s[st] > t[tt]) {
22.                 ans += 10;
```

```

23.         st--, tt--;
24.     } else {
25.         if (s[sh] < t[tt]) ans -= 10;
26.         sh++;
27.         tt--;
28.     }
29. }
30. cout << ans << endl;
31. }
32.
33. return 0;
34. }

```

2.3 看直播

描述

小鲁在业余时间的爱好之一是看直播，每个主播都有自己的直播时段，例如刘某的直播在7-10点直播。小鲁不能在一个时间段同时看两个直播，每看一个直播都需要从开头看到结尾。然而他有很多爱看的直播。怎样在同一时间内观看最多直播呢？

输入

输入数据包含多个测试实例，每个测试实例的第一行只有一个整数 n ($n \leq 100$)，表示你喜欢看的直播的总数，然后是 n 行数据，每行包括两个数据 Ti_s, Ti_e ($1 \leq i \leq n$)，分别表示第 i 个直播的开始和结束时间，为了简化问题，每个时间都用一个正整数表示。 $n=0$ 表示输入结束，不做处理。

输出

对于每个测试实例，输出能完整看到的直播的个数，每个测试实例的输出占一行。

输入样例 1

```

7
1 4
3 7
0 7
4 8
10 19
10 20
13 15
0

```

输出样例 1

3

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int N;
5. struct rg {
6.     int l, r;
7.     bool operator<(const rg &x) const { return r < x.r; }
8. } s[1000];
9.
10. int main() {
11.     while (~scanf("%d", &N) && N) {
12.         for (int a = 1; a <= N; a++) scanf("%d%d", &s[a].l, &s[a].r);
13.         sort(s + 1, s + 1 + N);
14.         int bd = -2147483637, ans = 0;
15.         for (int a = 1; a <= N; a++) {
16.             if (s[a].l < bd) continue;
17.             ans++;
18.             bd = s[a].r;
19.         }
20.         printf("%d\n", ans);
21.     }
22.
23.     return 0;
24. }
```

2.4 多多建设农场

描述

农民多多被选为他们镇的镇长！多多决定在镇上建立起互联网，并连接到所有的农场。

多多给他的农场安排了一条高速的网络线路，他想把这条线路共享给其他农场。

为了节省成本，多多想铺设最短的光纤去连接所有的农场。

多多得到了一份各农场之间连接费用的列表，现在需要找出能连接所有农场并所用光纤最短的方案。

输入

第一行：农场的个数， N ($3 \leq N \leq 100$)。

接下来的N行：一个N×N的矩阵,表示每个农场之间的距离dij。用空格分隔。（dij≤100000）

输出

能连接所有农场的所用光纤的最小长度

输入样例 1

```
4
0 4 9 21
4 0 8 17
9 8 0 16
21 17 16 0
```

输出样例 1

```
28
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int N, m[1000][1000], ans = 0;
5. int ok[1000], w[1000];
6.
7. int main() {
8.     cin >> N;
9.     memset(w, 0x3f, sizeof(w));
10.    for (int a = 1; a <= N; a++)
11.        for (int b = 1; b <= N; b++) cin >> m[a][b];
12.
13.    w[1] = 0;
14.    for (int t = 1; t <= N; t++) {
15.        int mv = 0x3f3f3f3f, mp = 0;
16.        for (int a = 1; a <= N; a++)
17.            if (!ok[a] && w[a] < mv) {
18.                mv = w[a];
19.                mp = a;
20.            }
21.        ok[mp] = 1;
22.        ans += mv;
23.        for (int b = 1; b <= N; b++)
24.            if (!ok[b] && w[b] > m[mp][b]) {
25.                w[b] = m[mp][b];
26.            }
27.    }
```

```
28. cout << ans << endl;
29.
30. return 0;
31. }
```

2.5 多多维持治安

描述

多多统领着 N 个部队，这 N 个部队分别驻扎在 N 个不同的城市。

他在用这 N 个部队维护着 M 个城市的治安，这 M 个城市分别编号从1到 M 。

现在，军师告诉多多，第 K 号城市发生了暴乱，多多将军从各个部队都派遣了一个分队沿最近路去往暴乱城市平乱。

现在已知在任意两个城市之间的路行军所需的时间，请你告诉多多第一个分队到达叛乱城市所需的时间。

输入

第一行：输入一个整数 T ，表示测试数据的组数。（ $T < 20$ ）

每组测试数据的第一行：四个整数 N, M, P, Q ($1 \leq N \leq 100, N \leq M \leq 1000, M-1 \leq P \leq 100000$) 其中 N 表示部队数， M 表示城市数， P 表示城市之间的路的条数， Q 表示发生暴乱的城市编号。

随后的一行是 N 个整数，表示部队所在城市的编号。

再之后的 P 行，每行有三个正整数， a, b, t ($1 \leq a, b \leq M, 1 \leq t \leq 100$)，表示 a, b 之间的路如果行军需要用时为 t

数据保证暴乱的城市是可达的。

注意，两个城市之间可能不只一条路。

输出

对于每组测试数据，输出第一支部队到达叛乱城市时的时间。

每组输出占一行

输入样例 1

```
1
3 8 9 8
1 2 3
1 2 1
2 3 2
1 4 2
2 5 3
3 6 2
```

```
4 7 1
5 7 3
5 8 2
6 8 2
```

输出样例 1

```
4
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int T;
5. int N, M, P, Q;
6. int hv_y[1010];
7. int e_num, la[1010], to[200020], nx[200020], wi[200020];
8. void add(int u, int v, int t) {
9.     to[++e_num] = v;
10.    nx[e_num] = la[u];
11.    la[u] = e_num;
12.    wi[e_num] = t;
13. }
14. int dis[1010], ok[1010];
15.
16. int Dij(int ben) {
17.    memset(dis, 0x3f, sizeof(dis));
18.    memset(ok, 0, sizeof(ok));
19.    dis[ben] = 0;
20.    for (int t = 1; t <= M; t++) {
21.        int mv = 0x3f3f3f3f, mp = 0;
22.        for (int a = 1; a <= M; a++)
23.            if (!ok[a] && mv > dis[a]) {
24.                mv = dis[a];
25.                mp = a;
26.            }
27.        if (!mp) break;
28.        ok[mp] = 1;
29.        for (int nxt, eg = la[mp]; eg; eg = nx[eg]) {
30.            nxt = to[eg];
31.            if (ok[nxt]) continue;
32.            dis[nxt] = min(dis[nxt], mv + wi[eg]);
33.        }
34.    }
35.
```

```

36.  int ans = 0x3f3f3f3f;
37.  for (int a = 1; a <= M; a++)
38.      if (hv_y[a]) ans = min(ans, dis[a]);
39.  return ans;
40. }
41.
42. int main() {
43.     scanf("%d", &T);
44.     while (T--) {
45.         scanf("%d%d%d%d", &N, &M, &P, &Q);
46.
47.         memset(hv_y, 0, sizeof(hv_y));
48.         for (int a = 1; a <= N; a++) {
49.             int t;
50.             scanf("%d", &t);
51.             hv_y[t] = 1;
52.         }
53.
54.         e_num = 0;
55.         memset(la, 0, sizeof(la));
56.         for (int a = 1; a <= P; a++) {
57.             int u, v, t;
58.             scanf("%d%d%d", &u, &v, &t);
59.             add(u, v, t);
60.             add(v, u, t);
61.         }
62.
63.         printf("%d\n", Dij(Q));
64.     }
65.
66.     return 0;
67. }

```

2.6 小H的选课计划

描述

小H在厦大本科期间总共要选 n 门必修课, 他可以自由决定这 n 门课的选课顺序.

然而, 学校规定一些课是有前置课程要求的. 比如要选机器学习, 就要先要学完高等数学, 线性代数和概率论.

另外, 小H对每门课都有一个兴趣值, 兴趣值越大表示他对这门课越感兴趣 (为了方便我们假设每门课程的兴趣值是唯一的).

请你帮小H设计一个选课计划,在满足学校规定的前提下优先选兴趣值高的课程。

输入

第一行是一个整数 $T[1,10]$,表示 T 组样例

对于每个样例:

第一行是一个整数 $n[10,30]$,表示有 n 门必修课,编号 $1\sim n$

第二行有 n 个数,第 i 个数表示课程 i 的兴趣值 $a_i[1,100]$,保证每个 a_i 是唯一的

接下有 n 行,其中第 i 行有一个整数 $k[0,n-1]$,表示课程 i 有 k 门前置课程,随后是 k 个前置课程
 $id[1,n]$

输出

输出 T 行

每行是 n 个课程 id ,表示最优的选课顺序, n 个数之间用空格隔开(注意最后一个数后没有空格)

输入样例 1

```
1
10
1 2 3 4 5 6 7 8 9 10
0
0
0
0
2 1 2
1 4
3 4 5 6
3 1 3 6
3 4 6 7
4 4 6 8 9
```

输出样例 1

```
4 6 3 2 1 8 5 7 9 10
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int N, T;
5. int s[1000];
6. int m[100][100];
7. int du[100], ok[100];
8.
9. int main() {
```



```

10.  cin >> T;
11.  while (T--) {
12.      cin >> N;
13.
14.      memset(du, 0, sizeof(du));
15.      memset(m, 0, sizeof(m));
16.      memset(ok, 0, sizeof(ok));
17.
18.      for (int a = 1; a <= N; a++) cin >> s[a];
19.
20.      for (int a = 1; a <= N; a++) {
21.          int k;
22.          cin >> k;
23.          for (int b = 1; b <= k; b++) {
24.              int t;
25.              cin >> t;
26.              m[t][a] = 1;
27.              du[a]++;
28.          }
29.      }
30.      int ini = 0;
31.      while (1) {
32.          int mv = -1, mp = 0;
33.          for (int a = 1; a <= N; a++)
34.              if (du[a] == 0 && !ok[a] && mv < s[a]) {
35.                  mv = s[a];
36.                  mp = a;
37.              }
38.          if (mv == -1)
39.              break;
40.          else {
41.              if (!ini)
42.                  ini = 1;
43.              else
44.                  printf(" ");
45.          }
46.          cout << mp;
47.          ok[mp] = 1;
48.          for (int a = 1; a <= N; a++)
49.              if (m[mp][a] && !ok[a]) du[a]--;
50.      }
51.      cout << endl;
52.  }
53.

```

```
54.  return 0;  
55. }
```

第3章 算法设计与分析实验C

3.1 小H保护小草

描述

每逢下雨,小H最喜欢的三叶草地就会积一潭水.这意味着小草可能被淹死.为了保护小草,小H修建了一套排水系统将草地的水排到小溪中,从而使草地免除被大水淹没的烦恼.

小H知道每一条排水沟的最大流量以及排水系统的准确布局.对于给出的每条排水沟,雨水只能沿着一个方向流动,注意可能会出现雨水环形流动的情形.

根据这些信息,请你帮助小H计算从水潭排水到小溪的最大流量。

输入

第一行:两个用空格分开的整数 M, N ($0 < M \leq 200, 0 < N \leq 200$). M 是小H已经挖好的排水沟的数量, N 是排水沟交叉点的数量. 交点1是草地, 交点 N 是小溪。

第二行到第 $M+1$ 行:每行有三个整数 S_i, E_i, C_i . S_i 和 E_i ($1 \leq S_i, E_i \leq N$) 指明排水沟两端的交点, 雨水从 S_i 流向 E_i . C_i ($0 < C_i \leq 100000$) 是这条排水沟的最大流量。

输出

输出一个整数, 即排水的最大流量。

输入样例 1

```
5 4
1 2 40
1 4 20
2 4 20
2 3 30
3 4 10
```

输出样例 1

```
50
```

提示

题型: 最大流问题

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3. const int Lnf = 1e9;
```

```

4. struct Dinic {
5. #define MXN 1300
6. #define MXM 10001 // 2M
7. #define lon int
8.     int S, T, N;
9.     struct Edge {
10.         int nx, to;
11.         lon f;
12.     } e[MXM];
13.     int e_num, la[MXN], cr[MXN];
14.     void init(int n, int s, int t) {
15.         N = n;
16.         S = s;
17.         T = t;
18.         e_num = 1;
19.         memset(la, 0, sizeof(int) * (N + 1));
20.     }
21.     void add(int fr, int to, lon f) {
22.         e[++e_num].to = to;
23.         e[e_num].nx = la[fr];
24.         la[fr] = e_num;
25.         e[e_num].f = f;
26.         e[++e_num].to = fr;
27.         e[e_num].nx = la[to];
28.         la[to] = e_num;
29.         e[e_num].f = 0;
30.     }
31.     int q[MXN], hd, tl, dep[MXN];
32.     int bfs() {
33.         memset(dep, 0, sizeof(int) * (N + 1));
34.         memset(cr, 0, sizeof(int) * (N + 1));
35.         hd = 0;
36.         dep[q[tl = 1] = S] = 1;
37.         while (++hd <= tl) {
38.             int now = q[hd];
39.             for (int nxt, eg = la[now]; eg; eg = e[eg].nx) {
40.                 nxt = e[eg].to;
41.                 if (dep[nxt] || !e[eg].f) continue;
42.                 if (!cr[now]) cr[now] = eg;
43.                 dep[nxt] = dep[now] + 1;
44.                 q[++tl] = nxt;
45.                 if (nxt == T) return 1;
46.             }
47.         }

```

```

48.     return 0;
49. }
50. lon dfs(int now, lon lef) {
51.     if (now == T || !lef) return lef;
52.     lon res = lef, i;
53.     for (int &eg = cr[now]; eg; eg = e[eg].nx) {
54.         if (dep[now] + 1 != dep[e[eg].to] || !e[eg].f) continue;
55.         i = dfs(e[eg].to, min(res, e[eg].f));
56.         if (i) e[eg].f -= i, e[eg ^ 1].f += i, res -= i;
57.     }
58.     return lef - res;
59. }
60. lon deal() {
61.     lon ans = 0, f;
62.     while (bfs())
63.         while (f = dfs(S, Lnf)) ans += f;
64.     return ans;
65. }
66. } d;
67.
68. int K, N, M;
69.
70. int main() {
71.     cin >> M >> N;
72.     d.init(200, 1, N);
73.     for (int a = 1; a <= M; a++) {
74.         int f, t, c;
75.         cin >> f >> t >> c;
76.         d.add(f, t, c);
77.     }
78.
79.     printf("%d\n", d.deal());
80.
81.     return 0;
82. }

```

3.2 小H的导师选择

描述

x大学研究生新生开学，需要进行研究生导师的分配：

- 1.每个老师都有多个心仪的新生。

2.每个老师最终只能担任一名新生的研究生导师。

3.每个新生只能分配一名老师作为研究生导师。

小H是该活动的志愿者，现在小H收集到了所有导师的意向。

问：小H要如何进行导师的分配，使得最多数量的老师能够担任心仪学生的导师

输入

输入数据包含多个测试实例：

每个测试实例的第一行是三个整数 k, m, n ，分别表示可能的组合数目，导师的人数，学生的人数。

($0 < k \leq 1000, 1 \leq m, n \leq 500$)

接下来包含 k 行，每行有两个数，表示老师 A_i 愿意担任新生 B_i 的导师。

输出

对于每个测试实例，输出一个整数，表示能够找到心仪学生的最多导师数。

输入样例 1

```
7 4 4
1 1
1 2
1 3
2 1
2 4
3 1
4 2
0
```

输出样例 1

```
4
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3. const int Lnf = 1e9;
4. struct Dinic {
5. #define MXN 13000
6. #define MXM 100100 // 2M
7. #define lon int
8. int S, T, N;
```

```

9.  struct Edge {
10.     int nx, to;
11.     lon f;
12. } e[MXM];
13. int e_num, la[MXN], cr[MXN];
14. void init(int n, int s, int t) {
15.     N = n;
16.     S = s;
17.     T = t;
18.     e_num = 1;
19.     memset(la, 0, sizeof(int) * (N + 1));
20. }
21. void add(int fr, int to, lon f) {
22.     e[++e_num].to = to;
23.     e[e_num].nx = la[fr];
24.     la[fr] = e_num;
25.     e[e_num].f = f;
26.     e[++e_num].to = fr;
27.     e[e_num].nx = la[to];
28.     la[to] = e_num;
29.     e[e_num].f = 0;
30. }
31. int q[MXN], hd, tl, dep[MXN];
32. int bfs() {
33.     memset(dep, 0, sizeof(int) * (N + 1));
34.     memset(cr, 0, sizeof(int) * (N + 1));
35.     hd = 0;
36.     dep[q[tl = 1] = S] = 1;
37.     while (++hd <= tl) {
38.         int now = q[hd];
39.         for (int nxt, eg = la[now]; eg; eg = e[eg].nx) {
40.             nxt = e[eg].to;
41.             if (dep[nxt] || !e[eg].f) continue;
42.             if (!cr[now]) cr[now] = eg;
43.             dep[nxt] = dep[now] + 1;
44.             q[++tl] = nxt;
45.             if (nxt == T) return 1;
46.         }
47.     }
48.     return 0;
49. }
50. lon dfs(int now, lon lef) {
51.     if (now == T || !lef) return lef;
52.     lon res = lef, i;

```

```

53.     for (int &eg = cr[now]; eg; eg = e[eg].nx) {
54.         if (dep[now] + 1 != dep[e[eg].to] || !e[eg].f) continue;
55.         i = dfs(e[eg].to, min(res, e[eg].f));
56.         if (i) e[eg].f -= i, e[eg ^ 1].f += i, res -= i;
57.     }
58.     return lef - res;
59. }
60. lon deal() {
61.     lon ans = 0, f;
62.     while (bfs())
63.         while (f = dfs(S, Lnf)) ans += f;
64.     return ans;
65. }
66. } d;
67.
68. int K, N, M;
69.
70. int main() {
71.     while (cin >> K) {
72.         if (!K) break;
73.         cin >> M >> N;
74.         d.init(1200, 1, 1200);
75.         for (int a = 1; a <= K; a++) {
76.             int l, r;
77.             cin >> l >> r;
78.             d.add(l + 1, r + 505, 1);
79.         }
80.         for (int a = 1; a <= M; a++) d.add(1, a + 1, 1);
81.         for (int a = 1; a <= N; a++) d.add(a + 505, 1200, 1);
82.         printf("%d\n", d.deal());
83.     }
84.
85.     return 0;
86. }

```

3.3 少少种谷物

描述

少少开辟了两块巨大的耕地A和B（你可以认为容量是无穷）想拿来种谷物。

她一共有 n 种谷物的种子，每种谷物的种子有1个，编号为1到 n 。

现在，第 i 种作物种植在A中种植可以获得 a_i 的收益，在B中种植可以获得 b_i 的收益。

注意，神奇的是，某些作物共同种在一块耕地中可以获得额外的收益，少少找到了规则中共有 m 种谷物组合，第 i 个组合中的作物共同种在A中可以获得 $c(1, i)$ 的额外收益，共同种在B中可以获得 $c(2, i)$ 的额外收益。

少少想知道，种植的最大收益是多少。

输入

第一行1个整数 n ，表示谷物种数。

第二行 n 个整数，表示 a_i 。

第三行 n 个整数，表示 b_i 。

第四行1个整数 m ，表示组合种数。

接下来 m 行中，第 i 行第一个整数 k_i ，表示第 i 个作物组合中的谷物种数，接下来两个整数 $c(1, i)$ 和 $c(2, i)$ ，然后 k_i 个整数，表示该组合中的谷物编号。

输出

种植的最大收益。

输入样例 1

```
3
4 2 1
2 3 2
1
2 3 2 1 2
```

输出样例 1

```
11
```

输入样例 2

```
4
4 6 7 3
3 8 1 4
2
3 4 5 1 2 3
2 3 1 1 4
```

输出样例 2

```
27
```

提示

题型：最小割问题

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3. const int Lnf = 1e9;
4. struct Dinic {
5.     #define MXN 13000
6.     #define MXM 100001 // 2M
7.     #define lon long long
8.     int S, T, N;
9.     struct Edge {
10.         int nx, to;
11.         lon f;
12.     } e[MXM];
13.     int e_num, la[MXN], cr[MXN];
14.     void init(int n, int s, int t) {
15.         N = n;
16.         S = s;
17.         T = t;
18.         e_num = 1;
19.         memset(la, 0, sizeof(int) * (N + 1));
20.     }
21.     void add(int fr, int to, lon f) {
22.         e[++e_num].to = to;
23.         e[e_num].nx = la[fr];
24.         la[fr] = e_num;
25.         e[e_num].f = f;
26.         e[++e_num].to = fr;
27.         e[e_num].nx = la[to];
28.         la[to] = e_num;
29.         e[e_num].f = 0;
30.     }
31.     int q[MXN], hd, tl, dep[MXN];
32.     int bfs() {
33.         memset(dep, 0, sizeof(int) * (N + 1));
34.         memset(cr, 0, sizeof(int) * (N + 1));
35.         hd = 0;
36.         dep[q[tl = 1] = S] = 1;
37.         while (++hd <= tl) {
38.             int now = q[hd];
39.             for (int nxt, eg = la[now]; eg; eg = e[eg].nx) {
40.                 nxt = e[eg].to;
41.                 if (dep[nxt] || !e[eg].f) continue;
42.                 if (!cr[nxt]) cr[nxt] = eg;
```

```

43.         dep[nxt] = dep[now] + 1;
44.         q[++tl] = nxt;
45.         if (nxt == T) return 1;
46.     }
47. }
48. return 0;
49. }
50. lon dfs(int now, lon lef) {
51.     if (now == T || !lef) return lef;
52.     lon res = lef, i;
53.     for (int &eg = cr[now]; eg; eg = e[eg].nx) {
54.         if (dep[now] + 1 != dep[e[eg].to] || !e[eg].f) continue;
55.         i = dfs(e[eg].to, min(res, e[eg].f));
56.         if (i) e[eg].f -= i, e[eg ^ 1].f += i, res -= i;
57.     }
58.     return lef - res;
59. }
60. lon deal() {
61.     lon ans = 0, f;
62.     while (bfs())
63.         while (f = dfs(S, Lnf)) ans += f;
64.     return ans;
65. }
66. } d;
67.
68. int N, M;
69. int ai[10000], bi[10000];
70. long long tot;
71.
72. int main() {
73.     scanf("%d", &N);
74.     for (int a = 1; a <= N; a++) scanf("%d", &ai[a]), tot += ai[a];
75.     for (int a = 1; a <= N; a++) scanf("%d", &bi[a]), tot += bi[a];
76.     scanf("%d", &M);
77.     d.init(M + M + N + 2, 1, 2);
78.     for (int a = 1; a <= N; a++) d.add(1, a + 2, ai[a]), d.add(a + 2, 2, bi[a]);
79.     for (int a = 1; a <= M; a++) {
80.         int ma, mb, k;
81.         scanf("%d%d%d", &k, &ma, &mb);
82.         tot += ma;
83.         tot += mb;
84.         d.add(1, 2 + N + a, ma);
85.         d.add(2 + N + M + a, 2, mb);
86.

```

```

87.     for (int b = 1; b <= k; b++) {
88.         int t;
89.         scanf("%d", &t);
90.         d.add(2 + N + a, 2 + t, Lnf);
91.         d.add(2 + t, 2 + N + M + a, Lnf);
92.     }
93. }
94.
95. printf("%lld\n", tot - d.deal());
96.
97. return 0;
98. }

```

3.4 多多的棋盘游戏

描述

多多想出了一款可以在棋盘上玩的新游戏。

棋盘上有 n 枚棋子，多多和少少进行操作，先手可以任意取走一枚棋子，之后每名游戏者取走的棋子都必须与上回合对手取走的棋子的曼哈顿距离不超过 L 。

(x_i, y_i) 和 (x_j, y_j) 之间的曼哈顿距离为 $|x_i - x_j| + |y_i - y_j|$ 。

在游戏开始时，少少开场，可以选择去除棋盘上的任何石头。

多多和少少很聪明，因此他们都使用最佳策略来玩这个游戏。

现在，多多想知道他是否可以确保赢得比赛。也就是，给出 L 以及 n 枚棋子的坐标，问是否后手必胜。

输入

第一行：输入case数；

接下来，在每种case下，第一行是一个正整数 n ($n \leq 361$)，棋子数量。

以下是 n 行，每行包含一对整数 x 和 y ($0 \leq x, y \leq 18$)，表示棋子坐标。

所有对都是不同的。最后一行是整数 L ($1 \leq L \leq 36$)。

每个case之间有空行

输出

如果多多可以赢得比赛，则输出“ YES”；否则，仅输出“ NO”。

输入样例 1

```

2
2
0 2
2 0

32

```

```
2  
  
2  
0 2  
2 0  
4
```

输出样例 1

```
NO  
YES
```

提示

题型：一般图匹配问题。

算法思路

```
1. #include <bits/stdc++.h>  
2. using namespace std;  
3. typedef long long lon;  
4. const lon Lnf = 0x3f3f3f3f3f3f;  
5. struct Dinic {  
6. #define MXN 2100  
7. #define MXM 10010 // 2M  
8. int S, M, N, T;  
9. struct Edge {  
10. bool operator<(Edge t1) { return f < t1.f; }  
11. int nx, to, fr;  
12. lon f;  
13. } e[MXM];  
14. struct edge {  
15. bool operator<(edge t1) { return f < t1.f; }  
16. int nx, to, fr;  
17. lon f;  
18. } E[MXM];  
19.  
20. int e_num, la[MXN], cr[MXN];  
21. void init(int s, int t) {  
22. S = s;  
23. T = t;  
24. e_num = 1;  
25. memset(la, 0, sizeof(int) * (N + 1));  
26. for (int j = 1; j <= M; j++) add(E[j].fr, E[j].to, E[j].f);  
27. }  
28. void add(int fr, int to, lon f) {
```

```

29.     e[++e_num].to = to;
30.     e[e_num].nx = la[fr];
31.     la[fr] = e_num;
32.     e[e_num].f = f;
33.     e[++e_num].to = fr;
34.     e[e_num].nx = la[to];
35.     la[to] = e_num;
36.     e[e_num].f = 0;
37. }
38.
39. int q[MXN], hd, tl, dep[MXN];
40. int bfs() {
41.     memset(dep, 0, sizeof(int) * (N + 1));
42.     memset(cr, 0, sizeof(int) * (N + 1));
43.     hd = 0;
44.     dep[q[tl = 1] = S] = 1;
45.     while (++hd <= tl) {
46.         int now = q[hd];
47.         for (int nxt, eg = la[now]; eg; eg = e[eg].nx) {
48.             nxt = e[eg].to;
49.             if (dep[nxt] || !e[eg].f) continue;
50.             if (!cr[now]) cr[now] = eg;
51.             dep[nxt] = dep[now] + 1;
52.             q[++tl] = nxt;
53.             if (nxt == T) return 1;
54.         }
55.     }
56.     return 0;
57. }
58. lon dfs(int now, lon lef) {
59.     if (now == T || !lef) return lef;
60.     lon res = lef, i;
61.     for (int &eg = cr[now]; eg; eg = e[eg].nx) {
62.         if (dep[now] + 1 != dep[e[eg].to] || !e[eg].f) continue;
63.         i = dfs(e[eg].to, min(res, e[eg].f));
64.         if (i) e[eg].f -= i, e[eg ^ 1].f += i, res -= i;
65.     }
66.     return lef - res;
67. }
68. lon deal() {
69.     lon ans = 0, f;
70.     while (bfs())
71.         while (f = dfs(S, Lnf)) {
72.             ans += f;

```

```

73.     }
74.     return ans;
75. }
76.
77. void csh() {
78.     cin >> M >> N;
79.     for (int i = 1; i <= M; i++) scanf("%d%d%d", &E[i].fr, &E[i].to, &E[i].f);
80.     init(1, N);
81.     cout << deal();
82. }
83. } dinic;
84. int main() {
85.     dinic.csh();
86.     return 0;
87. }

```

3.5 整数划分

描述

一个正整数 n 可以表示成若干个正整数之和，形如：

$n = n_1 + n_2 + \dots + n_k$ ，其中 $n_1 \geq n_2 \geq \dots \geq n_k, k \geq 1$ 。

我们将这样的一种表示称为正整数 n 的一种划分。

现在给定一个正整数 n ，请你求出 n 共有多少种不同的划分方法。

数据范围

$1 \leq n \leq 1000$

输入

共一行，包含一个整数 n 。

输出

共一行，包含一个整数，表示总划分数量。

由于答案可能很大，输出结果请对 10^9+7 取模。

输入样例 1

5

输出样例 1

7

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. const int Mod = 1E9 + 7;
5. int N;
6. long long f[1005][1005];
7.
8. long long dfs(int lef, int eeg) {
9.     if (lef == 0) return 1;
10.    if (f[lef][eeg]) return f[lef][eeg];
11.    f[lef][eeg] = 0;
12.    for (int a = min(eeg, lef); a; a--)
13.        f[lef][eeg] = (f[lef][eeg] + dfs(lef - a, a)) % Mod;
14.    return f[lef][eeg];
15. }
16.
17. int main() {
18.    memset(f, 0, sizeof(f));
19.    cin >> N;
20.    printf("%lld\n", dfs(N, N));
21.
22.    return 0;
23. }
```


第4章 算法设计与分析实验D

4.1 最短编辑距离

描述

给定两个字符串A和B，现在要将A经过若干操作变为B，可进行的操作有：

删除-将字符串A中的某个字符删除。

插入-在字符串A的某个位置插入某个字符。

替换-将字符串A中的某个字符替换为另一个字符。

现在请你求出，将A变为B至少需要进行多少次操作。

数据范围

$1 \leq n, m \leq 1000$

输入

第一行包含整数n，表示字符串A的长度。

第二行包含一个长度为n的字符串A。

第三行包含整数m，表示字符串B的长度。

第四行包含一个长度为m的字符串B。

字符串中均只包含大写字母。

输出

输出一个整数，表示最少操作次数。

输入样例 1

```
10
AGTCTGACGC
11
AGTAAGTAGGC
```

输出样例 1

```
4
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int N, M;
5. char s[1010], t[1010];
```

```

6. int f[1010][1010];
7.
8. int main() {
9.     cin >> N >> s + 1 >> M >> t + 1;
10.    memset(f, 0x3f, sizeof(f));
11.    f[0][0] = 0;
12.
13.    for (int a = 1; a <= N; a++)
14.        for (int b = 1; b <= M; b++) {
15.            f[a][b] = min(min(f[a - 1][b] + 1, f[a][b - 1] + 1),
16.                           f[a - 1][b - 1] + (s[a] != t[b]));
17.        }
18.    cout << f[N][M] << endl;
19.
20.    return 0;
21. }

```

4.2 小H的塔防游戏

描述

小H正在玩一款塔防游戏,需要在 $n \times n$ 的地图上布置 n 个防御塔(防御塔是相同的)拦截敌人
然而,防御塔的火力太强,可能波及到其他防御塔
具体而言,防御塔会攻击到同一行/同一列/同一斜线上的其他防御塔
请你帮助小H计算有多少种布阵方案,使得防御塔不会相互攻击

输入

一个整数 n ($1 \leq n \leq 8$),表示防御塔数量

输出

输出方案数

输入样例 1

4

输出样例 1

2

输入样例 2

8

输出样例 2

92

算法思路

```
1. #include <algorithm>
2. #include <cmath>
3. #include <iomanip>
4. #include <iostream>
5. #include <map>
6. #include <queue>
7. #include <set>
8. #include <string>
9. using namespace std;
10. int a[15] = {0};
11. int arr[15][15] = {0};
12. int sum = 0, n, b[1000], c[1000], d[1000];
13. void dfs(int deep) {
14.     if (deep == 0) {
15.         sum++;
16.         return;
17.     }
18.     for (int i = 1; i <= n; i++) {
19.         int l = n - deep + 1;
20.         if (b[i] == 0 && c[l + i] == 0 && d[l - i + n] == 0) {
21.             a[l] = i;
22.             b[i] = 1;
23.             c[l + i] = 1;
24.             d[l - i + n] = 1;
25.             dfs(deep - 1);
26.             a[l] = 0;
27.             b[i] = 0;
28.             c[l + i] = 0;
29.             d[l - i + n] = 0;
30.         }
31.     }
32.     return;
33. }
34.
35. int main() {
36.     cin >> n;
37.     dfs(n);
38.     cout << sum;
39.     return 0;
```

4.3 小H的塔防游戏2

描述

小H在你的帮助下成功通过了塔防游戏1.

现在来到了第二关,有一个 $n \times n$ 的地图,地图中'X'代表障碍物,'.'代表空地.

小H需要在空地上建防御塔,规定任意两个防御塔不能位于同一行或者同一列,除非它们之间有障碍物
请你帮助小H计算最多能建多少个防御塔

输入

第一行一个 n ($2 \leq n \leq 4$),表示地图的大小

4×4 的矩形,矩形的每个元素为'X'或'.' ,分别表示障碍物和空地

输出

输出防御塔的最大个数

输入样例 1

```
4
.X..
....
XX..
....
```

输出样例 1

```
5
```

算法思路

```
1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int N;
5. char s[10][10];
6. int pc[10][10] = {};
7. int ans = 0;
8.
9. int check(int u, int v) {
10.     int r, c;
11.     r = u;
12.     c = v;
```

```

13. while (--r && s[r][c] == '.')
14.     if (pc[r][c]) return 0;
15.     r = u;
16.     c = v;
17. while (++r <= N && s[r][c] == '.')
18.     if (pc[r][c]) return 0;
19.     r = u;
20.     c = v;
21. while (--c && s[r][c] == '.')
22.     if (pc[r][c]) return 0;
23.     r = u;
24.     c = v;
25. while (++c <= N && s[r][c] == '.')
26.     if (pc[r][c]) return 0;
27. return 1;
28. }
29.
30. void dfs(int u, int v, int dep) {
31.     if (v > N) {
32.         u++;
33.         v = 1;
34.     }
35.     if (u == N + 1) {
36.         ans = max(ans, dep);
37.         return;
38.     }
39.
40.     if (s[u][v] == 'X')
41.         dfs(u, v + 1, dep);
42.     else {
43.         if (check(u, v)) {
44.             pc[u][v] = 1;
45.             dfs(u, v + 1, dep + 1);
46.         }
47.         pc[u][v] = 0;
48.         dfs(u, v + 1, dep);
49.     }
50. }
51.
52. int main() {
53.     cin >> N;
54.     for (int a = 1; a <= N; a++) cin >> s[a] + 1;
55.
56.     dfs(1, 1, 0);

```

```
57.  
58.     cout << ans << endl;  
59.  
60.     return 0;  
61. }
```

4.4 小H的超大背包

描述

小H有一个容量特别大的背包,用来装一些体积很大的物品.

每个物品都有一个价值,并且只有一件

请你帮助小H计算这个超大背包最大能装多少价值的物品,并且这些物品的总体积不超过背包容量

输入

第一行两个整数 N, V ($1 \leq N \leq 8, 1 \leq V \leq 2000000000$),表示物品的数量和背包的容量

接下来 N 行:

每行有两个整数 v, w ($1 \leq v \leq 1000000000, 1 \leq w \leq 1000000$),表示每件物品的体积和价值

输出

输出最大价值

输入样例 1

```
4 5  
1 2  
2 4  
3 4  
4 5
```

输出样例 1

```
8
```

提示

N 很小,可以考虑回溯

算法思路

```
1. #include <algorithm>  
2. #include <cmath>  
3. #include <iostream>  
4. using namespace std;  
5.  
  
42
```

```

6.  const int N = 15;
7.  long long v[N], w[N];
8.  long long res = 0;
9.  long long bagv;
10. int n;
11.
12. void dfs(int st, long long used, long long value) {
13.     if (used > bagv || st > n) return;
14.     res = max(res, value);
15.     dfs(st + 1, used + v[st], value + w[st]);
16.     dfs(st + 1, used, value);
17. }
18. int main() {
19.     cin >> n >> bagv;
20.     for (int i = 0; i < n; i++) cin >> v[i] >> w[i];
21.     dfs(0, 0, 0);
22.     cout << res;
23. }

```

4.5 小H的旅行计划

描述

z王国有若干个城市,每两个城市之间都有一条道路。

由于z王国的特殊政策,每个城市只允许游客访问一次。

假设城市从0开始编号,请你帮助小H规划一条最短的旅行路线,使得小H从0到n-1不重不漏地访问每个城市

输入

第一行是一个整数 n ($1 \leq n \leq 8$),表示城市的个数

接下来是一个 $n \times n$ 的对称矩阵,元素 $dis[i, j]$ 表示 i 城市与 j 城市间道路的长度
($1 \leq dis[i, j] \leq 10000, dis[i, i] = 0, dis[i, j] = dis[j, i]$)

输出

输出最短的距离

输入样例 1

```

5
0 2 4 5 1
2 0 6 5 3
4 6 0 8 3

```

```
5 5 8 0 5
1 3 3 5 0
```

输出样例 1

```
18
```

提示

可以使用动态规划, 不过 n 很小, 可以考虑回溯或分支界限

算法思路

```
1. #include <algorithm>
2. #include <cstdio>
3. #include <iostream>
4. #include <vector>
5. using namespace std;
6. int n, cw, bestw;
7. int dis[10][10];
8. int x[10], bestx[10];
9. const int MAX = 3e10;
10. void dfs(int i) {
11.     if (i == n - 1) {
12.         if (dis[x[n - 2]][x[n - 1]] != MAX && dis[x[n - 1]][x[n]] != MAX)
13.             if (cw + dis[x[n - 2]][x[n - 1]] + dis[x[n - 1]][x[n]] < bestw) {
14.                 bestw = cw + dis[x[n - 2]][x[n - 1]] + dis[x[n - 1]][x[n]];
15.                 for (int j = 1; j <= n; j++) bestx[j] = x[j];
16.             }
17.     } else {
18.         for (int j = i; j < n; j++)
19.             if (dis[x[i - 1]][x[j]] != MAX && cw + dis[x[i - 1]][x[j]] < bestw) {
20.                 swap(x[i], x[j]);
21.                 cw = cw + dis[x[i - 1]][x[i]];
22.                 dfs(i + 1);
23.                 cw = cw - dis[x[i - 1]][x[i]];
24.                 swap(x[i], x[j]);
25.             }
26.     }
27. }
28.
29. int main() {
30.     cin >> n;
31.     for (int i = 1; i <= n; i++) {
32.         x[i] = i;
33.         for (int j = 1; j <= n; j++) cin >> dis[i][j];
```



```

34.  }
35.  bestw = MAX;
36.  cw = 0;
37.  dfs(2);
38.  cout << bestw;
39.  return 0;
40. }

```

4.6 小H的旅行计划2

描述

Z王国有若干个城市(编号从1开始),每两个城市之间都有一条道路.

由于Z王国的特殊政策,每个城市只允许游客访问一次.

请你帮助小H规划一条旅行路线,使得小H可以访问所有城市,并且每个城市只访问一次

由于城市数量过多,小H放宽了要求,最终的路径长度不需要最短,只要优于小H的可接受值

即可

然而我们并不知道小H的可接受值是多少,因此我们能做的就是尽可能缩短路径长度

输入

第一行是一个样例编号 t ,没有实际作用,只需要原封不动输出即可

第二行是一个整数 n ($1 \leq n \leq 50$),表示城市的个数

接下来是一个 $n \times n$ 的对称矩阵,元素 $dis[i, j]$ 表示 i 城市与 j 城市间道路的长度

($1 \leq dis[i, j] \leq 10000$, $dis[i, i] = 0$, $dis[i, j] = dis[j, i]$)

输出

第一行输出样例编号 t

第二行输出 n 个空格分开的整数,代表你的尽可能短的路径

输入样例 1

```

0
16
0 12 6 18 9 7 2 15 2 10 2 3 14 3 8 14
12 0 7 17 13 15 10 21 10 5 24 22 13 23 22 20
6 7 0 22 11 21 13 22 24 21 25 17 16 3 3 23
18 17 22 0 20 25 12 4 4 1 16 11 3 7 7 21

```

```

9 13 11 20 0 19 20 15 23 19 19 1 4 7 24 1
7 15 21 25 19 0 4 6 13 21 25 16 12 21 12 23
2 10 13 12 20 4 0 17 8 8 16 9 21 17 2 21
15 21 22 4 15 6 17 0 8 6 21 11 12 19 23 5
2 10 24 4 23 13 8 8 0 7 14 19 20 24 20 4
10 5 21 1 19 21 8 6 7 0 6 25 18 2 2 4
2 24 25 16 19 25 16 21 14 6 0 10 8 1 25 22
3 22 17 11 1 16 9 11 19 25 10 0 3 22 2 12
14 13 16 3 4 12 21 12 20 18 8 3 0 18 20 8
3 23 3 7 7 21 17 19 24 2 1 22 18 0 3 25
8 22 3 7 24 12 2 23 20 2 25 2 20 3 0 19
14 20 23 21 1 23 21 5 4 4 22 12 8 25 19 0

```

输出样例 1

```

0
2 10 14 11 1 7 6 8 13 4 9 16 5 12 15 3

```

提示

每组样例都设定了一个可接受值, 你的解只要小于这个可接受值就可以通过. 采用任何方法都可以推荐尝试分支限界, 实在解不出来的话就贪心吧:-)

算法思路

```

1. #include <bits/stdc++.h>
2. using namespace std;
3.
4. int T;
5. int N;
6. vector<pair<int, int> > e[55];
7. int cnt;
8. int vis[55];
9.
10. int best;
11. int ph[55], np[55];
12.
13. void dfs(int now, int dep, int sum) {
14.     cnt++;
15.     if (cnt >= 1e7) return;
16.
17.     np[dep] = now;
18.
19.     if (dep == N) {
20.         if (sum < best) {
21.             best = sum;

```

```

22.     memcpy(ph, np, sizeof(ph));
23. }
24. return;
25. }
26. vis[now] = 1;
27. for (int a = 0; a < N - 1; a++) {
28.     int to = e[now][a].second, wi = e[now][a].first;
29.     if (!vis[to]) {
30.         dfs(to, dep + 1, sum + wi);
31.     }
32. }
33. vis[now] = 0;
34. }
35.
36. int main() {
37.     while (scanf("%d", &T) != EOF) {
38.         printf("%d\n", T);
39.
40.         cnt = 0;
41.         best = 1E9;
42.         memset(vis, 0, sizeof(vis));
43.
44.         scanf("%d", &N);
45.         for (int a = 1; a <= N; a++) {
46.             e[a].clear();
47.             for (int b = 1; b <= N; b++) {
48.                 int t;
49.                 scanf("%d", &t);
50.                 if (a != b) e[a].emplace_back(t, b);
51.             }
52.         }
53.
54.         for (int a = 1; a <= N; a++) sort(e[a].begin(), e[a].end());
55.
56.         for (int a = 1; a <= N; a++) dfs(a, 1, 0);
57.
58.         for (int a = 1; a < N; a++) printf("%d ", ph[a]);
59.         printf("%d\n", ph[N]);
60.     }
61.
62.     return 0;
63. }

```

后记

1. 本实验教程的所有代码由codeinword提供格式转换: <http://www.codeinword.com/>
2. 本实验教程的所有代码在VSCode下编辑, 使用Devcpp编译通过, 并且在xmuoj.com上全部AC。
3. 本实验教程所有的题目都可以在<http://xmuoj.com>上在线练习。
4. 爱算法的同学, 笔者推荐ACWing (<https://www.acwing.com>)。要提前预备社招和面试的同学推荐, 笔者推荐力扣 (<https://leetcode-cn.com>)。
5. 本实验教程为算法设计与分析实验团队撰写的第二本讲义, 虽然经过团队成员的多轮校稿, 但是难免有错误和不足之处, 恳请见谅, 请发现本实验教程错误的同学邮件联系笔者, [反馈错误 37199625@qq.com](mailto:37199625@qq.com)。
6. 本实验教程的参考代码虽然能够AC题目, 但是并非每一个都是最优的代码, 我们把AC代码全部打印出来, 目的是起到抛砖引玉作用, 希望同学们AC题目后能够改进优化现有代码, 写出优雅效率更好的代码。如果事情如此成就, 那么笔者的班门弄斧也算是一桩美事。因此, 我们诚邀各位同学贡献自己写的更好的代码给我们, 让本实验教程的下一版AC代码能够更加有质量, 使后来人得着益处。